PYTHON FOR BEGINNERS · SESSION 12

Advanced Functions

Default and keyword arguments, *args, **kwargs, lambda, map and filter.

Recap

Ask the class:

- What is the difference between a **parameter** and an **argument**?
- What does a function with **no `return`** give back?
- Can a function read a **global** variable? Can it change one?

Default arguments

A parameter with a **default value** becomes **optional**. The caller can leave it out.

```
def welcome(name, greeting="Hello", punctuation="!"):
    return greeting + ", " + name + punctuation

print(welcome("Aman"))
print(welcome("Aman", "Good morning"))
print(welcome("Aman", "Hi", "?"))
```

OUTPUT

```
Hello, Aman!
Good morning, Aman!
Hi, Aman?
```

- Parameters **without** a default come **first**, then the ones **with** a default.
- The default is made **once**, when the function is **defined**.

The mutable default trap

A **list as a default** is shared by **every call**. It keeps growing!

Work out the output first, then press Run to check

```
def add_item(item, items=[]):
    items.append(item)
    return items

print(add_item("a"))
print(add_item("b"))
print(add_item("c"))
```

The safe way

Use **`None`** as the default and make the list **inside** the function.

```
def add_item(item, items=None):
    if items is None:
        items = []
    items.append(item)
    return items

print(add_item("a"))
print(add_item("b"))
print(add_item("c", ["x"]))
```

OUTPUT

```
['a']
['b']
['x', 'c']
```

Keyword arguments

Name the argument in the call: `name=value`. The **order no longer matters**, and the code is **easier to read**.

```
def book(title, author, year=2020):
    return f"{title} by {author} ({year})"

print(book("Python Basics", "Aman"))
print(book(author="Ankit", title="Data Tricks"))
print(book("Loops", "Kabir", year=2018))
```

OUTPUT

```
Python Basics by Aman (2020)
Data Tricks by Ankit (2020)
Loops by Kabir (2018)
```

- **Positional** arguments come **first**, **keyword** arguments after them.
- You cannot give the **same parameter twice**.

- Built-in functions use this: `print(..., sep="-", end="!")` and `sorted(..., reverse=True)`.

Try it: a positional argument after a keyword one

```
def book(title, author):
    return title + " by " + author

print(book(title="Python", "Aman"))
```

ERROR

```
  File "your code", line 4
    print(book(title="Python", "Aman"))
                                       ^
SyntaxError: positional argument follows keyword argument
```

*args: any number of arguments

A parameter with a **single star** collects **all the extra positional arguments** into a **tuple**. By habit it is called **`args`**.

```
def total(*args):
    print(args, type(args))
    result = 0
    for n in args:
        result += n
    return result

print(total(1, 2, 3))
print(total(10, 20, 30, 40))
print(total())
```

OUTPUT

```
(1, 2, 3) <class 'tuple'>
6
(10, 20, 30, 40) <class 'tuple'>
100
() <class 'tuple'>
0
```

Normal parameters plus *args

```
def show(first, *others):
    print("first:", first)
    print("others:", others)

show(1)
show(1, 2, 3)
```

OUTPUT

```
first: 1
others: ()
first: 1
others: (2, 3)
```

Ask

Which built-in function takes any number of arguments? Hint: you have used it since Session 1.

**kwargs: any number of keyword arguments

A parameter with a **double star** collects all the extra **keyword arguments** into a **dictionary**. By habit it is called **`kwargs`**.

```
def profile(**kwargs):
    print(kwargs, type(kwargs))
    for key, value in kwargs.items():
        print(key, "=", value)

profile(name="Aman", age=20, city="Delhi")
```

OUTPUT

```
{'name': 'Aman', 'age': 20, 'city': 'Delhi'} <class 'dict'>
name = Aman
age = 20
city = Delhi
```

All of them together

The **order** in the definition is: normal parameters, `*args`, then `**kwargs`.

```python
def demo(a, b=2, *args, **kwargs):
    print("a =", a)
    print("b =", b)
    print("args =", args)
    print("kwargs =", kwargs)

demo(1)
print()
demo(1, 5, 6, 7, x=10, y=20)
```

OUTPUT

```
a = 1
b = 2
args = ()
kwargs = {}

a = 1
b = 5
args = (6, 7)
kwargs = {'x': 10, 'y': 20}
```

Careful

The names `args` and `kwargs` are only a **habit**. The stars are what matter. `*numbers` and `**options` work just as well.

Unpacking when you call

The stars also work **in the call**. `*` spreads a list or tuple into **positional** arguments, `**` spreads a dictionary into **keyword** arguments.

```python
def volume(length, width, height):
    return length * width * height

sizes = [2, 3, 4]
print(volume(*sizes))

info = {"length": 5, "width": 2, "height": 3}
print(volume(**info))
```

OUTPUT

```
24
30
```

Passing everything on

```
nums = [5, 1, 9, 3]
print(max(*nums))
print(*nums)
print(*nums, sep=", ")

first = {"a": 1}
second = {"b": 2}
print({**first, **second})
```

```
OUTPUT
9
5 1 9 3
5, 1, 9, 3
{'a': 1, 'b': 2}
```

Keyword-only parameters

Parameters written **after `*args` (or after a bar `*`) can **only** be given by **name**. This stops mix-ups.

```
def connect(host, *, port=80, secure=False):
    return f"{host}:{port} secure={secure}"

print(connect("example.com"))
print(connect("example.com", port=443, secure=True))
```

```
OUTPUT
example.com:80 secure=False
example.com:443 secure=True
```

Try it: port must be named

```
def connect(host, *, port=80):
    return host + ":" + str(port)

print(connect("example.com", 443))
```

```
ERROR
  File "your code", line 4, in <module>
TypeError: connect() takes 1 positional argument but 2 were given
```

Functions are values

In Python a function is **a value like any other**. You can put it in a variable, a list, or **pass it to another function**.

```python
def shout(text):
    return text.upper() + "!"

say = shout
print(say("hello"))

def apply(func, value):
    return func(value)

print(apply(shout, "python"))
print(apply(len, "python"))
print(apply(abs, -7))
```

OUTPUT

```
HELLO!
PYTHON!
6
7
```

A list of functions

```python
def double(n):
    return n * 2

def square(n):
    return n * n

for func in [double, square, abs]:
    print(func(-3))
```

OUTPUT

```
-6
9
3
```

Ask

Why do we write `apply(shout, ...)` and not `apply(shout(), ...)`?

lambda: a function in one line

A **lambda** is a small **nameless function** written in one line: `lambda parameters: expression`. It **returns** the expression automatically.

```
square = lambda n: n * n
add = lambda a, b: a + b
greet = lambda name="friend": "Hello, " + name

print(square(6))
print(add(3, 4))
print(greet())
print(greet("Aman"))
```

OUTPUT

```
36
7
Hello, friend
Hello, Aman
```

`DEF` FUNCTION	`LAMBDA`
`def square(n): return n * n`	`lambda n: n * n`
has a **name**	is **nameless** (unless you store it)
many lines allowed	**one expression** only
uses `return`	returns the value **automatically**

When to use it

Use a `lambda` for a **tiny, one-time** function, usually **passed to another function**. For anything longer, write a normal `def`.

lambda with sorted, min and max

The **`key=`** argument takes a function that says **what to compare**. A `lambda` is perfect for it.

```
words = ["banana", "kiwi", "apple", "fig"]
print(sorted(words))
print(sorted(words, key=len))
print(sorted(words, key=lambda w: w[-1]))
print(max(words, key=len))
print(min(words, key=lambda w: w[1]))
```

OUTPUT

```
['apple', 'banana', 'fig', 'kiwi']
['fig', 'kiwi', 'apple', 'banana']
['banana', 'apple', 'fig', 'kiwi']
banana
banana
```

Sorting records

```
students = [("Aman", 85), ("Ankit", 92), ("Kabir", 78)]
print(sorted(students, key=lambda s: s[1]))
print(sorted(students, key=lambda s: s[1], reverse=True))

people = [{"name": "Aman", "age": 20}, {"name": "Ankit", "age": 19}]
print(sorted(people, key=lambda p: p["age"]))
```

OUTPUT

```
[('Kabir', 78), ('Aman', 85), ('Ankit', 92)]
[('Ankit', 92), ('Aman', 85), ('Kabir', 78)]
[{'name': 'Ankit', 'age': 19}, {'name': 'Aman', 'age': 20}]
```

map()

`map(function, items)` **applies a function to every item** and gives a **new sequence** of the results. Wrap it in `list()` to see the values.

```
nums = [1, 2, 3, 4]
print(list(map(lambda n: n * n, nums)))
print(list(map(str, nums)))
print(list(map(len, ["hi", "python", "a"])))
print(list(map(float, ["1.5", "2", "3.25"])))
```

OUTPUT

```
[1, 4, 9, 16]
['1', '2', '3', '4']
[2, 6, 1]
[1.5, 2.0, 3.25]
```

map with a named function

```
def to_fahrenheit(c):
    return c * 9 / 5 + 32

celsius = [0, 20, 37, 100]
print(list(map(to_fahrenheit, celsius)))
```

OUTPUT

```
[32.0, 68.0, 98.6, 212.0]
```

map with two lists

```
a = [1, 2, 3]
b = [10, 20, 30]
print(list(map(lambda x, y: x + y, a, b)))
```

OUTPUT

```
[11, 22, 33]
```

Careful

`map()` does **not** give a list. `print(map(...))` shows something like `<map object ...>`. Use `list(map(...))`.

filter()

`filter(function, items)` **keeps only the items** for which the function returns **True**.

```
nums = [1, 2, 3, 4, 5, 6, 7, 8]
print(list(filter(lambda n: n % 2 == 0, nums)))
print(list(filter(lambda n: n > 5, nums)))

words = ["apple", "", "kiwi", "", "fig"]
print(list(filter(None, words)))
print(list(filter(lambda w: len(w) > 3, words)))
```

OUTPUT

```
[2, 4, 6, 8]
[6, 7, 8]
['apple', 'kiwi', 'fig']
['apple', 'kiwi']
```

- The function must answer **True or False** for each item.
- `filter(None, items)` removes every **falsy** item: `0`, `""`, `None`, `[]`.
- Like `map`, it needs `list()` to show the result.

Passing marks

```
marks = {"Aman": 85, "Ankit": 42, "Kabir": 67, "Meera": 30}
passed = list(filter(lambda name: marks[name] >= 50, marks))
print(passed)
```

OUTPUT

```
['Aman', 'Kabir']
```

map and filter together

Chain them: **filter first**, then **map**.

```
nums = [1, 2, 3, 4, 5, 6]
even_squares = list(map(lambda n: n * n, filter(lambda n: n % 2 == 0, nums)))
print(even_squares)
```

OUTPUT

```
[4, 16, 36]
```

The same with a list comprehension

Many Python programmers prefer a **list comprehension**: it says the same thing and reads more clearly.

```
nums = [1, 2, 3, 4, 5, 6]
print([n * n for n in nums if n % 2 == 0])
print([n * n for n in nums])
print([n for n in nums if n > 3])
```

OUTPUT

```
[4, 16, 36]
[1, 4, 9, 16, 25, 36]
[4, 5, 6]
```

JOB	MAP / FILTER	COMPREHENSION
change every item	`map(f, items)`	`[f(x) for x in items]`
keep some items	`filter(f, items)`	`[x for x in items if f(x)]`

Ask

Which one is easier to read for you, the `map` / `filter` line or the comprehension?

Two more helpers: zip and reduce

`zip()` **pairs up** items from two lists. `functools.reduce()` **combines** all the items into **one value**.

```
names = ["Aman", "Ankit", "Kabir"]
marks = [85, 92, 78]
print(list(zip(names, marks)))
print(dict(zip(names, marks)))

for name, mark in zip(names, marks):
    print(name, mark)
```

OUTPUT

```
[('Aman', 85), ('Ankit', 92), ('Kabir', 78)]
{'Aman': 85, 'Ankit': 92, 'Kabir': 78}
Aman 85
Ankit 92
Kabir 78
```

```
from functools import reduce

nums = [1, 2, 3, 4, 5]
print(reduce(lambda a, b: a + b, nums))
print(reduce(lambda a, b: a * b, nums))
print(reduce(lambda a, b: a if a > b else b, nums))
```

OUTPUT

```
15
120
5
```

A function that returns a function

Since functions are values, a function can **build and return another function**. The inner one **remembers** the argument it was made with.

```
def make_multiplier(n):
    return lambda x: x * n

double = make_multiplier(2)
triple = make_multiplier(3)
print(double(5), triple(5))
print(make_multiplier(10)(4))
```

OUTPUT

```
10 15
40
```

> **Note**
> You do not need this every day, but you will see it in real code, for example in **decorators** (a later topic).

Common mistakes

- A **list or dictionary as a default** value: use `None` instead.
- A **positional argument after a keyword** argument: it is a `SyntaxError`.
- Writing a **long `lambda`**: if it is hard to read, use `def`.
- Printing `map(...)` or `filter(...)` **without `list()`**.
- Calling the function inside `map`: `map(f(), items)` is wrong, write `map(f, items)`.
- A `lambda` for `filter` that **returns a value** but not a **True or False** meaning.

Try it: what is wrong here?

```
nums = [1, 2, 3]
print(map(lambda n: n * 2, nums))
print(list(map(lambda n: n * 2, nums())))
```

```
ERROR
<map object at 0xd2cc80>
  File "your code", line 3, in <module>
TypeError: 'list' object is not callable
```

Practice

Warm-up questions

1. Write a function `power(base, exp=2)` and call it with and without the second argument.
2. Write a function `average(*nums)` that works for any number of numbers.
3. Write a function that takes `**kwargs` and prints each key and value on its own line.
4. Use `map` and a `lambda` to turn `[1, 2, 3]` into `["#1", "#2", "#3"]`.
5. Use `filter` and a `lambda` to keep the words longer than 3 letters.

Harder questions

6. Sort a list of `(name, marks)` tuples by marks, highest first.
7. Use `map` and `filter` to print the squares of the odd numbers from 1 to 10.
8. Write `make_adder(n)` that returns a function adding `n` to its argument.
9. Write a function `describe(name, *hobbies, **details)` and call it in two different ways.
10. Rewrite your `map` and `filter` answers as list comprehensions.

Find the output

Work out the output first, then press Run to check

```
def f(a, *args, **kwargs):
    return a, args, kwargs

print(f(1))
print(f(1, 2, 3, x=4))
nums = [3, 8, 5, 10]
print(list(filter(lambda n: n > 4, nums)))
print(list(map(lambda n: n // 2, nums)))
print(sorted(nums, key=lambda n: -n))
```

Try the lambda question

Your turn: print the names sorted by length, then press Run

```
names = ["Meera", "Aman", "Kabir", "Ria"]
# sort by the length of each name
```

Homework and wrap-up

- Write a function `join_words(*words, sep=" ")` that joins any number of words.
- Write a function that takes a list of numbers and returns the **squares of the even numbers** using `filter` and `map`, then again with a comprehension.
- Given a list of dictionaries with `name` and `price`, print them **sorted by price** using a `lambda`.
- Write a function `make_greeter(greeting)` that returns a function, and use it to make `hello` and `namaste`.
- Revise: **defaults**, **keyword arguments**, `*args`, `**kwargs`, `lambda`, `map`, `filter`.

Next class

Files and error handling.